Image Filtering using data level parallelism

Group 1's Members:

- Catapang, Nathan James
- Garzon, Harris Martin
- Ruiz, Joseph Benjamin
- Tamayo, Chelsea Nichole

I. Abstract

Image filtering is a common operation in image processing and within the field of technology. Yet, it is not limited to just technology. Medical and industrial fields utilize this feature to reduce noise within images, clear and smooth out surroundings, and enhance visuals to perform optimally. However, applying filters to high-resolution images can involve substantial computational work due to the high amount of pixels needed to be processed. This process is usually done one at a time through sequential implementation, increasing the processing time as image resolution grows. This project aims to investigate the use of data-level parallelism to improve the performance of image filtering. The project will implement Sobel edge detection, using both sequential and parallel algorithms. The performance of both implementations will be evaluated using images with different resolutions, with execution time serving as the primary metrics.

II. Proposed Project Description

A. Inputs

The inputs will be images in PNG and JPEG formats. These images will be converted into grayscale before being processed. This is because the Sobel Edge detection algorithm detects edges by calculating changes in pixel intensity between neighboring pixels. Converting the images first will allow the algorithm to work with only one intensity value per pixel instead of processing the red, green, and blue values separately. This will make the calculations simpler and reduce the amount of data that needs to be processed. The project will also use images with different dimensions to compare the execution times of the sequential and SIMD implementations and see how image size affects their performance.

Each pixel will be represented by an 8-bit unsigned integer with an intensity value ranging from 0 to 255, where 0 represents black and 255 represents white. The pixel values will be stored in a one-dimensional array. The image width and height will be used to determine the position of each pixel so that both the sequential and SIMD implementations will use the same input data when performing the Sobel calculations. Using the same images will also help ensure a fair comparison of their execution times and check if both implementations produce the correct output.

B. Proposed Process

The project will focus on implementing the Sobel edge detection algorithm to identify the edges and boundaries within digital images. Firstly, the input image will be converted into grayscale to

simplify the process by representing each pixel using a single intensity value instead of separate red, green and blue values. This helps the sobel algorithm detect changes in pixel intensity more efficiently and make the edge detection process more straightforward. Afterwards, the algorithm will be applied to calculate the horizontal and vertical intensity gradients of each pixel using two (3x3) convolution kernels. These gradients will be combined using the gradient magnitude formula, $G = sqrt\{(G^{X\hat{}2} + G^{Y\hat{}2})\}$, to determine the strength of the edges in the image. Pixels with larger gradient magnitudes will indicate more prominent edges, which will be highlighted in the resulting output image.

To evaluate the benefits of data-level parallelism, the project will implement both sequential and parallel versions of the Sobel algorithm. The sequential implementation will serve as the baseline, processing the image pixel by pixel, while the parallel implementation will use AVX2 assembly instructions to process multiple pixels simultaneously. Research done by Jain et al. (2016), who investigated parallel Sobel edge detection using CUDA C and compared its performance with sequential execution. Although their implementation uses GPU-based parallelism, our project will explore CPU-based SIMD parallelism using AVX2. Their research provides reference for investigating parallel Sobel processing but does not establish the performance improvement AVX2 will achieve. Both implementations will use the same input images to ensure fair comparison. Their execution times will be measured and speedup will be calculated by dividing the sequential execution time by the parallel execution time across different image resolutions to evaluate performance.

C. Outputs

The main outputs of the project will be the transformed images saved back to the disk in the target output format, with detailed execution measurements. These give performance enhancement benchmarks contrasting sequential execution times against SIMD-accelerated execution times across various image resolutions.

The results will also include a comparison of the execution times and speedup of the SIMD implementation compared to the sequential implementation. Both implementations will be checked to make sure they produce the correct edge-detected images. The execution times and speedup will be presented in tables to make it easier to compare their performance across different image resolutions.

III. Target Platform

The project will use C for the sequential implementation and x86-64 AVX2 assembly language for the parallel implementation. The assembly kernel will be called from the C program. Development and testing will be performed on a computer that supports AVX2. GCC and NASM will be used to compile and execute the program. The computer used for testing must support AVX2 instructions to execute the parallel implementation.

Performance will be evaluated by comparing the execution times of both implementations across different image resolutions.

IV. Mid-Project Milestone

By week 10, the sequential C implementation and an initial version of the AVX2 assembly implementation of the Sobel edge detection algorithm should be completed. Both implementations will be tested using sample images to verify that they produce correct and comparable edge-detected outputs. Preliminary execution-time measurements will also be collected using images with different resolutions to assess the performance of the parallel implementation compared to the sequential baseline. Any errors or performance issues identified during testing will be addressed before the final demonstration.

V. References

Jain, A., Namdev, A., & Chawla, M. (2016). Parallel edge detection by SOBEL algorithm using CUDA C. *IEEE Students Conference on Electrical, Electronics and Computer Science*, 1–6. https://doi.org/10.1109/sceecs.2016.7509360

Moukthika. (2025, May 6). Edge detection using OpenCV. OpenCV. https://opencv.org/edge-detection-using-opencv/